

INTRODUZIONE A JAVA

PIETRO DI LENA

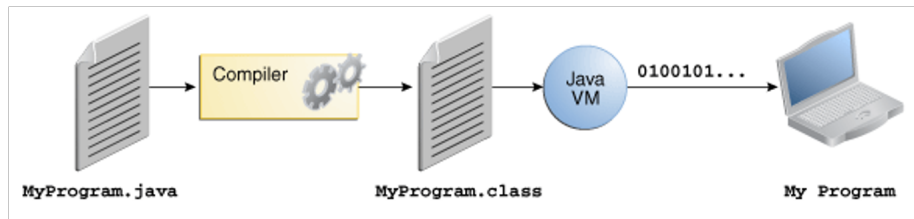
DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



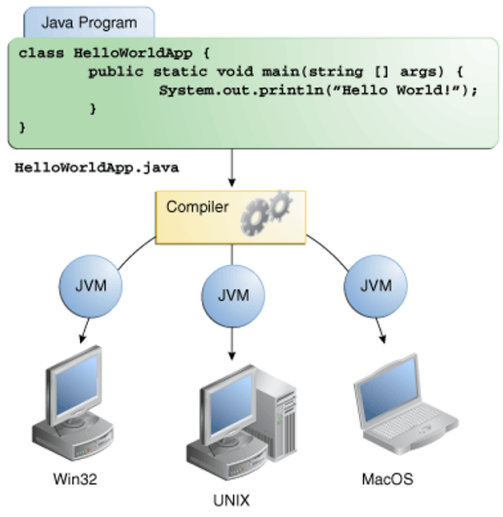
- Caratteristiche principali
 - Linguaggio di programmazione ad alto livello
 - Orientato agli oggetti
 - Sintassi molto simile a quella del C/C++
 - Sviluppato a partire dai primi anni 90 (prima release nel 1995)
 - Ha guadagnato popolarità con l'esplosione di internet
 - Particolarmente adatto per le applicazioni grafiche e di rete
- Obiettivi degli sviluppatori del linguaggio Java
 - semplice, orientato agli oggetti e familiare
 - robusto e sicuro
 - indipendente dalla piattaforma e portabile (WORA)
 - eseguibile con elevate prestazioni
 - interpretato, threaded e dinamico

LA PIATTAFORMA JAVA

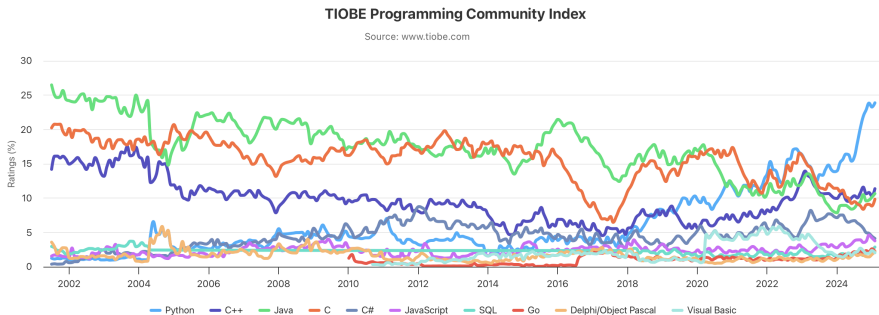


<http://docs.oracle.com/javase/tutorial/getStarted/intro/definition.html>

JAVA VIRTUAL MACHINE

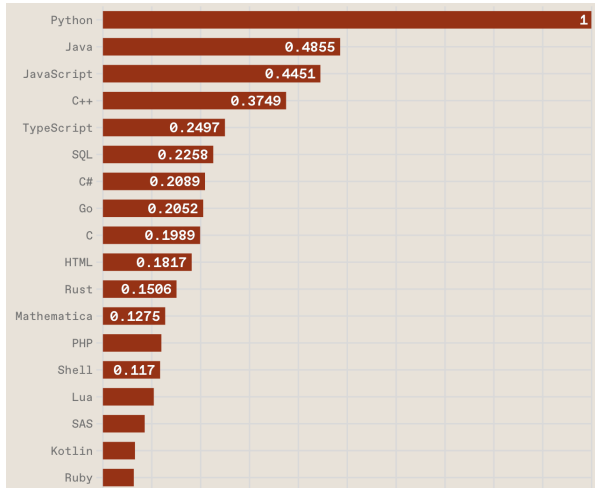


INDICE DI POPOLARITÀ DI JAVA (TIOBE)



<https://www.tiobe.com/tiobe-index/>

INDICE DI POPOLARITÀ DI JAVA (IEEE SPECTRUM)



<https://spectrum.ieee.org/top-programming-languages-2024>

- Tutorial su Java
 - Non è un corso esaustivo su Java o OOP
 - Tratteremo solo alcuni aspetti del linguaggio
 - Utilizzeremo Java per implementare algoritmi e strutture dati
 - Impareremo ad utilizzare librerie built-in ed esterne
- Conoscenze pregresse assunte
 - Programmazione con linguaggi procedurali
 - Nozioni di programmazione ad oggetti
- Ambiente di sviluppo
 - Quello che preferite
 - Preferenza del docente: terminale Unix
 - IDE suggerita: Eclipse (<http://www.eclipse.org/ide>)

HELLO WORLD IN JAVA

```
1 public class HelloWorld {  
2  
3     public static void main(String args[]) {  
4         System.out.println("Hello World");  
5     }  
6  
7 }
```

- Semplice programma che stampa "Hello World" sul terminale
- Il nome del file deve coincidere con il nome della classe:

HelloWorld.java

CLASSI E OGGETTI

- Un programma Java è organizzato come un insieme di **oggetti** che
 - contengono **variabili** (valori o oggetti)
 - contengono **metodi** (funzioni)
- Un **oggetto** è un'istanza di una **classe**
- Una **classe** è un **tipo di dato** definito dal programmatore
 - Definisce variabili e metodi degli oggetti di quella classe
 - Nasconde l'implementazione (**incapsulamento**)
- Una classe contiene una parte **pubblica** ed una **privata**
 - La parte pubblica è visibile da chi usa un'istanza della classe
 - La parte privata (incapsulata) non né visibile né utilizzabile
 - La **visibilità** di metodi e variabili è decisa dal programmatore

MODIFICATORI DI VISIBILITÀ

- Classi, variabili e metodi possono avere diversi livelli di visibilità
- Il livello di visibilità è modificato con apposite keyword
 - **public**: visibile ed utilizzabile da tutte le altre classi
 - **protected**: visibile ed utilizzabile solo da
 - sottoclassi della classe corrente (lo vedremo)
 - classi nello stesso *package* (lo vedremo)
 - **private**: visibile ed utilizzabile solo all'interno della stessa classe
- I livelli di visibilità permettono di nascondere i dettagli implementativi
- E' compito del programmatore decidere gli opportuni livelli di visibilità

MODIFICATORI DI ACCESSO: FINAL

- **final**: indica che il metodo/variabile/classe è immutabile
- Una variabile *final* è una variabile costante

```
1 final int x = 10;  
2 x = 0; // Errore
```

- Una classe *final* non può essere **estesa** (lo vedremo)
- Un metodo *final* non può essere **sovrascritto** (lo vedremo)

MODIFICATORI DI ACCESSO: STATIC

- **static**: indica che esiste solo una istanza del metodo/variabile
- Una variabile *static* è condivisa da tutti gli oggetti istanziati

```
1 public class Counter {  
2     static int count = 0;  
3     ...  
4 }
```

Tutti gli oggetti di tipo *Counter* condivideranno *count*

- Un metodo *static* è condiviso da tutti gli oggetti istanziati

```
1 public class Message {  
2     static void print() {  
3         System.out.println("Ciao");  
4     }  
5     ...  
6 }
```

Possiamo invocare *print()* senza istanziare *Message*: *Message.print()*

- Possiamo avere anche blocchi e classi *static* (non lo vediamo)

ISTANZIARE UNA CLASSE

- Il **costruttore** è un particolare metodo di una classe
 - Ha lo stesso nome della classe
 - Non ha un tipo di ritorno
 - Il livello di visibilità è sempre public
 - Ammette parametri
 - E' possibile specificare diversi costruttori con parametri differenti
 - Viene utilizzato per inizializzare lo stato interno dell'oggetto
- Se omesso, la classe acquisisce un costruttore vuoto di default
 - Il costruttore non prende parametri e non fa nulla
- La keyword **new** permette di istanziare un nuovo oggetto
- Il costruttore viene invocato (automaticamente) utilizzando new

HELLO WORLD AD OGGETTI

```
1 public class HelloWorld {  
2     private final String msg;  
3  
4     public HelloWorld(String msg) {  
5         this.msg = msg;  
6     }  
7     public void print() {  
8         System.out.println(msg);  
9     }  
10    public static void main(String args[]) {  
11        HelloWorld msg = new HelloWorld("Hello, World");  
12        msg.print();  
13    }  
14 }
```

- *Message(String)*: **costruttore** della classe Message
- *msg*: **variabile** locale **privata** e **immutabile**
- *print()*: **metodo pubblico** della classe Message
- *new*: **keyword** per **istanziare** un oggetto della classe specificata

DOCUMENTAZIONE IN JAVA

```
1  /**
2   * OOP <b>HelloWorld</b> in Java
3   * @author Mario Rossi
4   * @version 1.0
5   */
6  public class HelloWorld {
7      private final String msg;
8      /**
9       * Constructs an object containing a message
10      * @param msg A string
11      */
12     public HelloWorld(String msg) {
13         this.msg = msg;
14     }
15     /**
16      * Prints the saved message
17      */
18     public void print() {
19         System.out.println(msg);
20     }
21     /**
22      * Main method
23      * @param args command line strings (unused)
24      */
25     public static void main(String args[]) {
26         HelloWorld msg = new HelloWorld("Hello, World");
27         msg.print();
28     }
29 }
```

[javadoc](#) genera documentazione HTML per la classe

ORGANIZZAZIONE FILE E PACKAGE

- Java adotta un approccio semplice per organizzare le classi
 - Ogni classe è in un file con stesso nome ed estensione *.java*
 - Le classi possono essere raggruppate in **packages**
 - Una classe appartenente al package *packagename* deve contenere

```
package packagename;
```

- Le classi nello stesso package devono essere nella stessa directory
 - Il nome della directory deve coincidere con il nome del package
 - I package possono essere organizzati in *subpackages*
 - Ogni *subpackage* deve essere nella sottodirectory corrispondente
- Esempio:
 - La classe *LinkedList* appartiene al package *datastructures.lists*
 - La classe *LinkedList* è contenuta nel file *LinkedList.java*
 - *LinkedList.java* si trova nella directory *datastructures/lists/*
 - Nel file *LinkedList.java* la prima istruzione deve essere

```
package datastructures.lists;
```


IMPORT E ACCESSO AI PACKAGE

- Possiamo referenziare direttamente solo classi nella stessa directory
- Per utilizzare classi in altre directory/package è necessario **importarle**
- Esempio:
 - Consideriamo la classe *BinaryTree* nel package *datastructures.trees*
 - Assumiamo che *BinaryTree* voglia utilizzare la classe *LinkedList*
 - Tale classe è nel package *datastructures.lists*
 - Le due classi sono in directory differenti (non si vedono)
 - *BinaryTree* può importare *LinkedList* con il seguente comando

```
import datastructures.lists.LinkedList;
```

- *BinaryTree* può anche importare tutte le classi nel package

```
import datastructures.lists.*;
```

- Java offre una vasta libreria di classi (da importare) già implementate
- Ad esempio, per utilizzare la classe [Color](#) dobbiamo usare il comando

```
import java.awt.Color;
```

OPERATORI E STRUTTURE DI CONTROLLO

- Sintassi simile (o perfettamente identica) a quella del C/C++ per:
 - Operatori (relazionali, logici, aritmetici, cast, assegnamento, ..)
 - Strutture di controllo condizionali (if, switch, ..)
 - Strutture di controllo iterative (for, while, ..)
 - Delimitazione blocchi (parentesi graffe { })
- Java non ammette espressioni *non definite*. Esempi:
 - $i = i++$ è non definito in C/C++ ma definito in Java
 - in $foo() + boo()$ l'ordine di valutazione degli operandi è definito in Java (da sinistra verso sinistra), non specificato in C/C++
- Non entreremo nel dettaglio di questi argomenti
 - Se e quando necessario, li vedremo tramite esempi/esercizi

TIPDI DI DATO IN JAVA

- Java è un linguaggio fortemente tipizzato (strongly-typed)
 - Ogni espressione ha un tipo (come in C/C++)
- Il tipo stabilisce le operazioni e i valori possibili per un dato
 - Obbligatorio indicarlo quando si dichiara una variabile
 - Il compilatore lo usa per verificare la correttezza delle operazioni
- Abbiamo due classi principali di tipo di dato
 - **tipi primitivi**: built-in nel linguaggio (dimensione fissa)
 - **tipi classe**: definiti dal programmatore

TIPPI PRIMITIVI IN JAVA

Tipo	Bit	Default	Min	Max
boolean	1	false	false	true
byte	8	0	-128	127
char	16	'\u0000'	'\u0000' (0)	'\uffff' ($2^{16}-1$)
short	16	0	-2^{15}	$2^{15} - 1$
int	32	0	-2^{31}	$2^{31} - 1$
long	64	0L	-2^{63}	$2^{63} - 1$
float	32	0f	$\approx -3.4 \times 10^{38}$	$\approx 3.4 \times 10^{38}$
double	64	0d	$\approx -1.6 \times 10^{308}$	$\approx 1.6 \times 10^{308}$

- Ogni tipo di dato primitivo ha una dimensione fissa
- Ogni variabile ha un valore di default (se non inizializzata)

WRAPPER E (UN)BOXING AUTOMATICO

- In Java è spesso utile/obbligatorio utilizzare solo variabili oggetto
- I tipi primitivi non possono essere trattati come tipi di dato oggetto
- Java definisce una classe **wrapper** per ogni tipo primitivo
- Java gestisce automaticamente la conversione da
 - tipo primitivo e la classe wrapper corrispondente (**boxing**)
 - classe wrapper a tipo primitivo corrispondente (**unboxing**)

Tipo	Classe	Creazione	Accesso
boolean	Boolean	obj = Boolean.valueOf(true)	obj.booleanValue()
byte	Byte	obj = Byte.valueOf((byte)10)	obj.byteValue()
char	Character	obj = Character.valueOf('a')	obj.charValue()
short	Short	obj = Short.valueOf((short)10)	obj.shortValue()
int	Integer	obj = Integer.valueOf(10)	obj.intValue()
long	Long	obj = Long.valueOf(10L)	obj.longValue()
float	Float	obj = Float.valueOf(10f)	obj.floatValue()
double	Double	obj = Double.valueOf(10.0)	obj.doubleValue()

Nota. L'uso di **new** per allocare l'oggetto è possibile ma deprecato

ESEMPIO CLASSI WRAPPER

```
1 Integer a = 8; // boxing
2 int b = Integer.valueOf(2); // unboxing
3 Integer c = Integer.parseInt("-10");
4 Double d = Double.parseDouble("3.14");
5
6 Double sum = a + b + c + d; // boxing and unboxing
7
8 System.out.println(sum);
```

Le conversioni (un)boxing sono automatiche nella somma e assegnamento

ALTRE CLASSE UTILI

- **String**: classe per memorizzare/manipolare stringhe

- Esempio: concatenazione con operatore +

```
1 String name      = new String("Mario");  
2 String surname = new String("Rossi");  
3 String person  = name + " " + surname;
```

- Esempio: calcolo lunghezza con metodo `.length()`

```
person.length();
```

- Numerosi metodi per manipolare stringhe (vedi [String](#))

- **enum**: classe speciale per le enumerazioni

- Esempio: semi di carte da gioco

```
1 public enum Seme {CUORI, QUADRI, PICCHE, FIORI};
```

- I valori elencati sono oggetti e possono essere usati come variabili

```
2 Seme s = Seme.CUORI;
```

ARRAY IN JAVA

- Un array (vettore) è un contenitore di:
 - dimensione fissa (per ridimensionarlo bisogna ri-allocarlo)
 - elementi tutti dello stesso tipo
- Sintassi per inizializzare un vettore in Java

```
1 Integer[] X = new Integer[10];  
2 int[] Y = new int[10];  
3 String[] Z = {"A", "B", "C", "D"};  
4  
5 System.out.println(X.length + Y.length + Z.length);
```

- La classe [Arrays](#) espone metodi (statici) per manipolare array

ESERCIZIO

- Implementare in java una classe *Persona* per memorizzare:
 - Nome
 - Cognome
 - Età
 - Continente di residenza
- Il continente deve essere implementato tramite enumerazione
Asia, Africa, America, Antartide, Europa, Oceania
- La classe *Persona* deve implementare quattro metodi
 - Costruttore (anche più di uno, con valori di default)
 - *getInfo()*: ritorna una stringa con informazioni, ad es.
Mi chiamo Mario Rossi ho 20 anni e vivo in Europa
 - *setAge()*: modifica l'età
 - *setContinent()*: modifica il continente di residenza